國立臺北科技大學
National Taipei University of Technology

# 資料結構

# Data Structure

# Lab08

姓名：林彥廷

學號：112370219

| Lab01-Q1. |
| --- |
| 1. Modify the inorder traversal function to a postorder traversal function in Part1.<br>2. Implement a method to find the maximum value in the left and right subtrees.<br>3. Explain how to find the maximum value in the left and right subtrees.<br>4. What happens if the left and right subtrees are not complete trees? |

**Code**

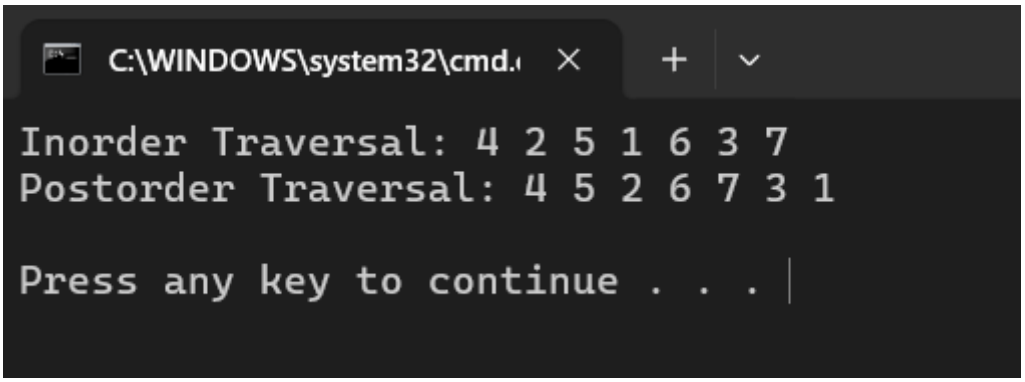
```
#include <iostream>
#include <queue>
#include <vector>
#include <climits>
#include <algorithm>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;              // 節點的值
    TreeNode* left;         // 左子節點
    TreeNode* right;        // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節
點
};
// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入 queue

        size_t i = 1; // 陣列索引
```

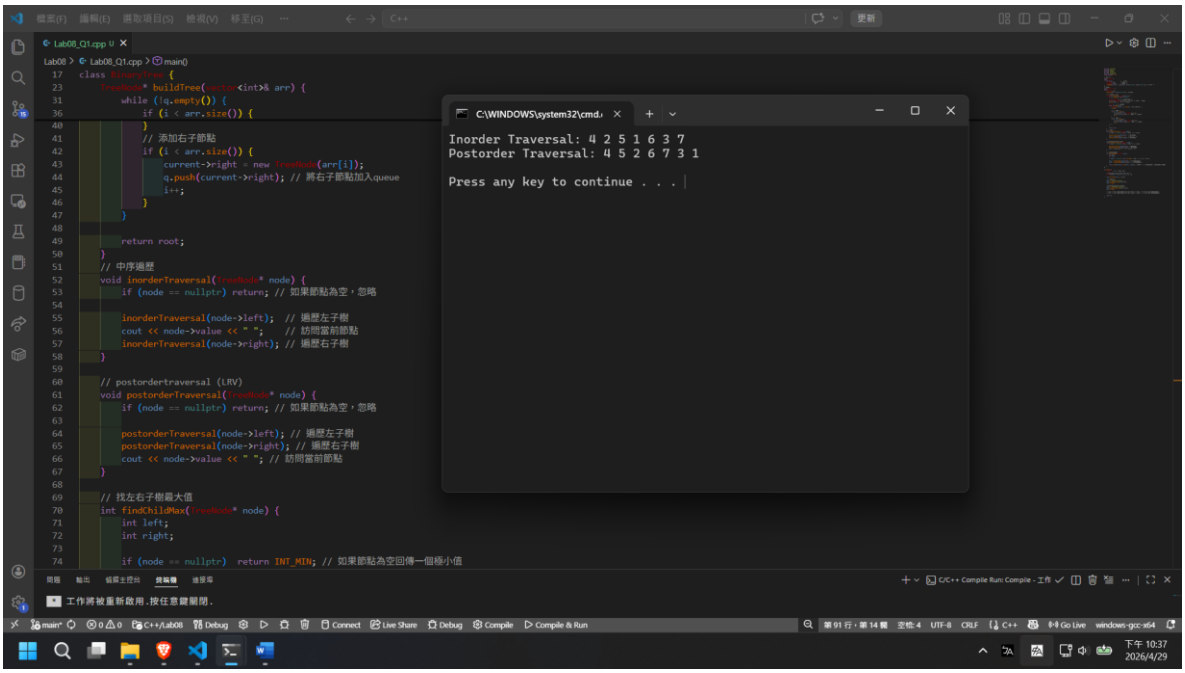
```
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
            if (i < arr.size()) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left); // 將左子節點加入 queue
                i++;
            }
            // 添加右子節點
            if (i < arr.size()) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入 queue
                i++;
            }
        }

        return root;
    }
    // 中序遍歷
    void inorderTraversal(TreeNode* node) {
        if (node == nullptr) return; // 如果節點為空，忽略

        inorderTraversal(node->left);  // 遍歷左子樹
        cout << node->value << " ";    // 訪問當前節點
        inorderTraversal(node->right); // 遍歷右子樹
    }

    // postordertraversal (LRV)
    void postorderTraversal(TreeNode* node) {
        if (node == nullptr) return; // 如果節點為空，忽略

        postorderTraversal(node->left); // 遍歷左子樹
        postorderTraversal(node->right); // 遍歷右子樹
        cout << node->value << " "; // 訪問當前節點
    }
```

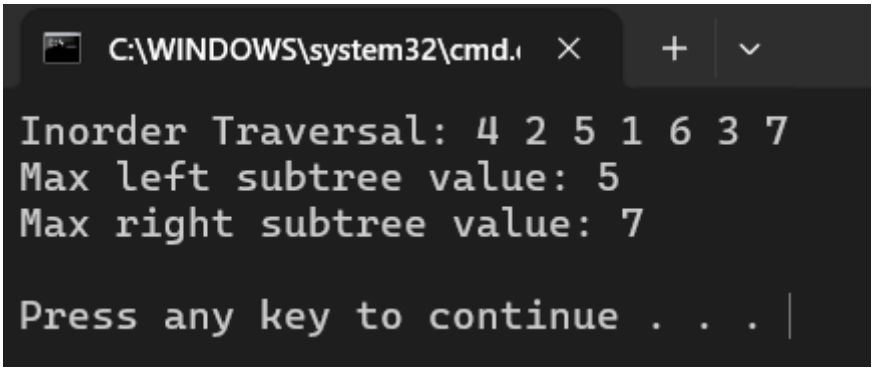
```
    // 找左右子樹最大值
    int findChildMax(TreeNode* node) {
        int left;
        int right;

        if (node == nullptr)  return INT_MIN; // 如果節點為空回傳一個極小值

        left = findChildMax(node->left); // 尋找左子樹最大值
        right = findChildMax(node->right); // 尋找右子樹最大值

        return max(node->value, max(left, right)); // 比較當前節點, 左子樹最大值,
右子樹最大值誰最大
    }
};

int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹，NULL 表示空子節點
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

    tree.buildTree(arr); // 建立樹

    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;

    // postordertraversal outcome
    cout << "Postorder Traversal: ";
    tree.postorderTraversal(tree.root);
    cout << endl;

    cout << tree.findChildMax(tree.root->left) << endl; // 尋找 root node 的左子
樹最大值
    cout << tree.findChildMax(tree.root->right) << endl; // 尋找 root node 的右子
樹最大值
```

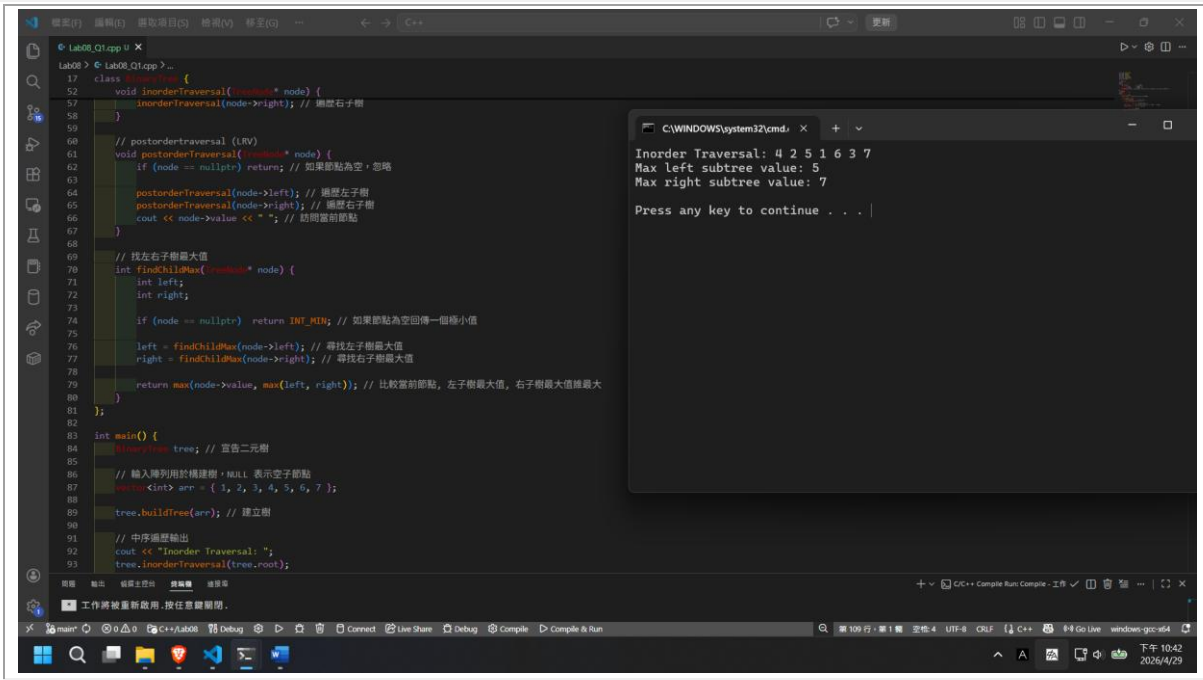
```
    return 0;
}
```

**Result**






## Discussion

1. 設計一個 findChildMax 函式：走到每個節點時，先問左子樹「你那邊最大值多少？」、再問右子樹「你那邊最大值多少？」，然後把左邊答案、右邊答案、自己的值，三個拿來比大小，回傳最大的。如果左右子樹還有 child ，就繼續往下，直到 leaf node。在 main 函式裡只需要對根的左子樹和右子樹各呼叫一次這個函式，就分別得到左子樹最大值和右子樹最大值。
2. 不管樹長什麼形狀 findChildMax 不受影響，遇到 nullptr 就回傳 INT_MIN，遞迴照樣處理。